

PRACTICAL

1. Virtual Address

```
GNU nano 8.7.1 pgm.c
#include <stdio.h>

int main() {
int x = 10;
printf("Value of x = %d\n", x);
printf("Virtual address of x = %p\n", (void *)&x);
return 0;
}
```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano pgm.c
abhis@MIKHAIL-37:~$ gcc pgm.c -o pgm
abhis@MIKHAIL-37:~$ ./pgm
Value of x = 10
Virtual address of x = 0x7ffdf0746404
```

2. Address Space

```
GNU nano 8.7.1 ww.c
#include <stdio.h>
#include <stdlib.h>
int global = 10;
int bss_variable;
int main() {
int local = 20;
int *heap = malloc(sizeof(int));
*heap = 30;
printf("Code address : %p\n", (void *)main);
printf("Data address : %p\n", (void *)&global);
printf("BSS address : %p\n", (void *)&bss_variable);
printf("Heap address : %p\n", (void *)heap);
printf("Stack address : %p\n", (void *)&local);
free(heap);
return 0;
}
```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano ww.c
abhis@MIKHAIL-37:~$ gcc ww.c -o ww
abhis@MIKHAIL-37:~$ ./ww
Code address : 0x5b5395a7c1a9
Data address : 0x5b5395a7f010
BSS address : 0x5b5395a7f018
Heap address : 0x5b53b79c3010
Stack address : 0x7fff674f021c
```

3.

```
GNU nano 8.7.1 er.c
#include <stdio.h>

int main() {
    int page_table[4] = {1, -1, 3, -1};
    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano er.c
abhis@MIKHAIL-37:~$ gcc er.c -o er
abhis@MIKHAIL-37:~$ ./er
Enter page number (0-3): 3
Page Fault!
Page must be loaded into memory.
```

4.

```

GNU nano 8.7.1 ZX.C
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number       = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}

```

OUTPUT :

```

abhis@MIKHAIL-37:~$ nano zx.c
abhis@MIKHAIL-37:~$ gcc zx.c -o zx
abhis@MIKHAIL-37:~$ ./zx
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number     = 1
Physical Address = 150

```

5. MallocFree

```

GNU nano 8.7.1 ll.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}

```

OUTPUT :

```

abhis@MIKHAIL-37:~$ nano ll.c
abhis@MIKHAIL-37:~$ gcc ll.c -o ll
abhis@MIKHAIL-37:~$ ./ll
10 20 30 40 50

```

6.

```

GNU nano 8.7.1 bh.c
#include <stdio.h>

int main() {
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");
    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano bh.c
abhis@MIKHAIL-37:~$ gcc bh.c -o bh
abhis@MIKHAIL-37:~$ ./bh
Before accessing memory
Segmentation fault      ./bh
```